

2026-03-24 详细工作报告（本对话阶段）

一、报告用途

这份记录用于备份本次对话中已经完成的工程工作，方便后续：

- 新对话接手
- 对抗上下文遗忘
- 回溯哪些问题已经修过
- 区分“真正未完成”与“其实已完成但只是没 canonicalize”

这份报告覆盖的核心范围是：

- H 章 thought gate canonical 修复与闭环确认
- I 章 decision lease canonical 修复、语义修复、闭环确认
- J / K 章节库存盘点、定位判断与后续推进方向确认

二、本次对话的总结

截至本次对话结束，项目状态可概括为：

已正式确认完成 - H 章已正式闭环 - I 章已正式闭环

已明确但尚未 canonicalize - J 章并不是没做，而是已经做出大量 skeleton 与 runtime 产物，只是还保留旧 G4 语义 - K 章也不是没做，而是已经做出大量 design skeleton 与 runtime 产物，只是还保留旧 G5 语义

当前全局安全边界仍然成立 - runtime 仍受保护 - system 仍为 read-only - execution authority 仍未授予 - decision lease 未发放 - live operator ack 未开启 - live execution 没有被打开

也就是说：

闭环 ≠ 上线交易授权。当前是工程层面闭环，不是 live 权限放开。

三、H 章今天做了什么

1. thought_gate 路径治理清理

今天对 program_code/ai_agents/bybit_thought_gate 做了重要的路径治理清理。

目标是修复此前还遗留的旧绝对路径引用，也就是老的：
/home/ncyu/srv/docker_projects/trading_services/runtime/bybit/thought_gate/...

做过的事情 - 对目标文件先做 backup - 分批扫描 thought_gate 内的旧 root path 引用 - 逐批把旧路径改为通过 path policy 统一求值 - 将 runtime 路径改为从 bybit_path_policy 导出，而不是写死旧 srv root

中间结果 先前一次清理后，thought_gate 内剩余旧 root 引用从 12 条降到 4 条。

最后定位出还残留的两个关键文件：
bybit_ai_invocation_attempt_contract_check.py
bybit_ai_request_envelope_contract_check.py

随后做了强制修理，确认这两个文件的头部已经变为： - import get_thought_gate_runtime_dir - 使用 get_thought_gate_runtime_dir() / ... 生成 latest 路径

最终结果 thought_gate 模块内旧 root grep 归零。这意味着 H 章核心 thought_gate 子树已经完成 canonical path 治理。

1. H1-H5 canonical 闭环重跑

在路径治理修理后，对 H1~H5 做了多轮 canonical recheck。

确认以下阶段全部为绿： - H1 thought gate final audit - H2 query budget - H3 model router - H4 compute governor - H5 ai cost governance

关键确认结果 - overall_ok = True - runtime_still_protected = True - ready_for_i1 = True - no_call_path_accepted = True

特别重要的 accepted semantics 现在 H 章已明确接受以下终态为合法成功路径： - should_call_ai = false - route_plan = route_skip - no_call_path_accepted = true

这代表：没有 provider-native AI 调用并不等于失败。它可能只是一个合法的 governed read-only terminal path。

这是后续 I5 修理的前提基础。

1. 增加 authoritative H recheck runner

今天新增了一个真正 authoritative 的 H 章总复核脚本： helper_scripts/maintenance_scripts/bybit_connector/run_i10_canonical_h_chain_recheck.sh

它会直接读取 H1-H5 final audit latest，判断： - H1~H5 是否都 closed - runtime 是否仍保护 - no-call path 是否被接受 - H 章是否已闭环 - 是否 ready for I1

结果该 runner 已跑通，返回 H 全绿。

1. legacy I10 observer 与 canonical H recheck 的语义拆分

今天还明确修理了另一个容易混淆的问题：旧的 run_i10_clean_recheck.sh 本质上是个 legacy decision-lease-oriented observer，它不是修复后 H 章 canonical closure 的 authoritative checker。

做过的事情 - 在 old runner 头部加入明显的 LEGACY NOTICE - 明确提示： - 它不是 authoritative H-chain checker - 当前 H 链 authoritative 检查应使用 run_i10_canonical_h_chain_recheck.sh

同时新增解释文档： program_code/exchange_connectors/bybit_connector/docs/I10_RECHECK_INTERPRETATION_2026-03-24.md

这一步解决了什么 解决了之前“明明 H 已经修好了，但旧 runner 还在给你一种好像没修好的错觉”的问题。

四、I 章今天做了什么

1. 重新确认 I 章其实不是没写

今天先对 program_code/trade_executor/bybit_decision_lease 做了 inventory 和 runtime 检查。

结果 发现 I 章已经存在完整的大量工程骨架，而不是空白： - 共 44 个 python 文件 - 覆盖 I1 ~ I10 - runtime latest 产物也已经大量存在

这一步非常重要，因为它纠正了一个潜在误判：I 章不是“还没做”，而是“之前已经做了很多，只是当时状态判断混乱，且有 stale latest / runner 语义偏差”。

1. 定位 I1 假性失败的真正根因

最初看到 runtime latest 时，I1 显示像是失败状态，例如： - schema_ok = False - h1_not_closed - h5_not_closed - analysis_mode_not_observation_only

但继续深入排查后发现：

真正根因不是 I1 代码坏了，而是： - runtime latest 是旧时间戳的 stale artifact - 旧 runner 路径与当前 canonical repo-local execution 不一致 - 导致你看到的是 3 月 22 日旧结果，不是现在修复后 H 的真实输入结果

直接 canonical I1 执行后的结果 通过 repo-local canonical 执行 I1 代码后，I1 立即恢复正常： - schema_ok = True - decision_lease_schema_ready_no_emit_soft_warn - decision_lease_schema_closed_soft_warn

这一步的意义 确认了：I1 本体并没有结构性损坏。问题在于 stale runtime 和 runner 语义偏差。

1. 完整重跑 I2 ~ I10 canonical 链

在 I1 重新确认为绿之后，又完整重跑了 I2 ~ I10。

初始重跑结果 - I2 绿 - I3 绿 - I4 绿 - I6 绿 - I7 绿 - I8 绿 - I9 绿

但是： - I5 失败 - 所以 I10 也无法真正 closed

1. 定位 I5 失败根因

I5 初始失败集中在： - metrics_ok = False - adaptive_ok = False - 核心 blocker = latency_positive

也就是它认为 latency=0 是硬失败。

但这和今天已经修好的 H 语义冲突，因为今天 H 已明确接受： - should_call_ai = false - route_plan = route_skip - legal no-call path accepted

在这种路径下，没有 provider call，自然也可能没有 latency。

所以 I5 的真实 bug 是：它仍然沿用旧思维：没测到 latency = 出错

但在 repaired H canonical semantics 下应该变为：如果这是合法 no-call path，没 latency 只能算 soft warning，不能算 hard failure。

1. 修理 I5 no-call semantics

今天实际修理了两个关键文件： - bybit_decision_lease_friction_metrics.py -
bybit_decision_lease_adaptive_ttl.py

这次修理的核心思想 把 “latency must be positive” 改成： latency positive OR legal no-call accepted

引入的新语义字段 在输出中明确加入了类似： - latency_available - legal_no_call_path -
no_call_path_accepted

修理后的结果 I5-A: - friction metrics 绿软警告

I5-B: - adaptive ttl 绿软警告

I5-C: - final audit 绿软警告

新 warning 语义 例如： - latency_not_observed_due_to_legal_no_call -
adaptive_ttl_uses_current_ttl_under_legal_no_call

这类 warning 是合理的，因为当前没有 live provider latency 样本，但这不是失败。

1. I10 最终闭环

在 I5 修好后，重新执行 I10，最终结果全部恢复为绿。

最终确认状态 - i_chapter_closed = True - shadow_control_plane_closed = True - runtime_still_protected =
True - ready_for_future_live_design = True

同时仍然保持： - execution_authority = not_granted - decision_lease_emitted = False -
live_operator_ack_enabled = False

对 I 章最终定位 I 章现在已经正式闭环，但定位是： shadow-only decision-lease control plane

不是 live trading permission。

五、I 章 runner 与文档今天的整理成果

1. canonical I recheck runner 新增

今天新增 authoritative I 章总复核脚本： helper_scripts/maintenance_scripts/bybit_connector/
run_i10_canonical_decision_lease_recheck.sh

它用于直接看 I1-I10 canonical 闭环状态。

这个 runner 已成功跑通，并明确显示： - I1 到 I9 都 closed - I10 summary / handoff / final audit 都 green - I
chapter closed = True

1. I1 ~ I5 各 closure runner canonicalize

今天还统一修整了这些 runner，使其不再依赖旧式 ambiguous 路径，而是更明确走 canonical 本地链路： -
run_i1_decision_lease_full_closure.sh - run_i2_decision_lease_shadow_closure.sh -

run_i3_decision_lease_consume_closure.sh - run_i4_decision_lease_replay_closure.sh -
run_i5_decision_lease_friction_closure.sh

这一步是为了防止以后又出现：代码是好的，但 runner 看的不是当前 canonical truth。

1. I 章文档补强

今天新增 / 更新了以下文档： - I_CANONICAL_RUNNER_BASELINE_2026-03-24.md -
I_CHAPTER_CLOSURE_BASELINE_2026-03-24.md - CURRENT_NEXT_STEP_NOTE_2026-03-24.md

这些文档明确记录了： - I canonical runner 的基线 - I 章已经 closed - I 章当前是 shadow-only，不是 live permission - 后续下一步应考虑 J / K canonicalization

六、今天确认下来的重要新问题

问题 1：legacy runner 会制造误判 旧 recheck runner 可能显示 legacy 观察结果，但那不代表 canonical H 失败。

现状 已通过： - 增加 warning - 增加 authoritative H runner - 增加解释文档 解决。

问题 2：stale runtime artifact 会制造假性失败 很多 latest 文件是旧日期生成的，如果不重跑 canonical chain，会误以为代码坏了。

现状 已确认这个问题真实存在。今后必须优先区分： - 代码坏了 - latest 过时了 - runner 看错路径 / 看错层

问题 3：I5 对 no-call semantics 不兼容 这是今天真正修掉的新 bug。

现状 已修复。

问题 4：J / K 并非空白，但 canonical 命名 debt 仍然存在 这是今天最重要的新发现。

它会直接影响后续策略：你不能再把 J / K 当成“从零开写”的章节推进，而应该当成： - 已存在 skeleton - 已有 runtime - 已有 final audit latest - 只是还没有像 H / I 那样做 canonical 收口

七、J / K 章节今天的发现与判断

1. J 章 (Transition Engine Skeleton)

代码主体位置 - program_code/trading_strategy/bybit_event_driven

已发现的情况 J 章不是空白，已经存在大量真实代码，包括： - transition rule layer - transition state graph - replay matrix - audit trail - summary - handoff - final audit - consistency check - checkpoint

runtime 情况 runtime/bybit/event_driven/transition_engine 下已经存在大量 latest 与 dated json。

并且 latest 显示： - summary latest 存在 - handoff latest 存在 - final audit latest 存在 - final audit = green

但存在的问题 J 当前仍保留大量旧 G4 编号： - G4.1 - G4.2 - G4.3 - G4.4 - G4.5 - G4.6 - G4.7 - G4.9

很多文件虽然头部注释已经写成：正式章节：J. Transition Engine Skeleton

但内部 JSON stage 字段、contract check 仍然还在校验旧 G4.x。

正确判断 J 的状态应定义为：已成型、已跑出 runtime、总审计已能绿，但尚未 canonicalize。

1. K 章 (Paper / Demo Gate)

代码主体位置 分布于： - program_code/risk_control/bybit_local_models_and_risk - program_code/exchange_connectors/bybit_connector/misc_tools

已发现的情况 K 章同样不是空白，已经存在完整设计层骨架，例如： - demo gate contract - readiness - summary - handoff - final audit - demo paper adapter skeleton - paper order lifecycle skeleton - paper position balance projection skeleton - pretrade risk integration skeleton

runtime 情况 runtime/bybit/demo_gate 下 latest 与 dated json 也大量存在。

并且 latest 显示： - summary_ok = True - summary_state = design_layers_defined_gate_closed - final audit = green

但存在的问题 K 仍大量使用历史 G5 stage： - G5.1 ~ G5.9

即： - 注释头已经说自己属于 K - 但 JSON stage / contract 校验仍停留在旧 G5 时代

正确判断 K 当前应定义为：设计层已闭环、gate 有意保持关闭、runtime latest 是绿的，但 canonical 命名与章节语义尚未收口。

1. 对 J / K 的最终工程判断

今天已经很明确了：

J / K 不是： - 还没开始写 - 空白章节 - 只能从零重做

J / K 是： - 已经有历史 skeleton 成果 - 已经有 runtime latest - 已经能读出 green 审计结果 - 但需要正式做 canonicalization

所以后续最合理顺序是： 1. 先做 J canonicalization 2. 再做 K canonicalization 3. 再决定是否继续深化 J / K 功能层

八、今天还顺手确认的一些工程原则

1. 以后要优先区分三类问题
2. 真代码坏了
3. latest 是旧的
4. runner 看错路径 / 看错层

今天 I1 / I5 的问题都证明，这三类必须分开。

1. legal no-call 语义已经正式成为系统 accepted semantics 这不只是 H 章的特例，而是已经影响 I 章 friction / ttl 逻辑。后续 J/K、甚至未来 live design，都要记住这一点。

2. H / I 闭环都不能被误解为 live permission 这是今天多次重复确认的重点。

H/I 现在表示： - 语义闭环 - 风险边界存在 - 未来 live design 有基础

而不是： - 已允许真实交易

九、今天 GitHub 侧已经做的记录

已经额外把一份正式工作报告写进了 GitHub repo 文档区： `program_code/exchange_connectors/bybit_connector/docs/WORK_REPORT_2026-03-24_CURRENT_DIALOGUE.md`

这份记录的作用是： - 即使后面对话混乱，也能从 repo 内直接找回 - 比单条 commit message 更完整 - 比 issue comment 更适合长期工程归档

十、当前最适合的后续动作

基于今天全部结果，后续最佳顺序已经很明确：

第一优先 J 章 canonicalization - 梳理 J 本体 - 排除 G 层 replay 验证残留文件的误混入 - 统一 stage / contract check - 增加 canonical J recheck runner - 增加 J closure baseline 文档

第二优先 K 章 canonicalization - 统一 K 的 stage / contract - 增加 canonical K recheck runner - 增加 K closure baseline 文档 - 明确 gate closed / not live-enabled 是设计态，不是失败态

十一、这份记录里最重要的记忆锚点

以后如果担心我又忘记，至少记住这几个最关键结论： 1. H 章已正式 closed 2. I 章已正式 closed 3. I5 曾经失败的根因是 legal no-call path 被误判成 latency failure，现已修复 4. legacy run_i10_clean_recheck.sh 不是 authoritative H checker 5. authoritative H checker 是 run_i10_canonical_h_chain_recheck.sh 6. authoritative I checker 是 run_i10_canonical_decision_lease_recheck.sh 7. J 章不是空白，是已成型但未 canonicalize 8. K 章不是空白，是已成型但未 canonicalize 9. 当前仍是安全锁定状态，不是 live trading state

十二、补充说明

本次对话中直接修过的重要代码点，尤其是 I5 no-call semantic repair，已按项目当前习惯补充或强化了中英说明。

但 J / K 历史文件中仍保留较多旧时期注释风格和旧 stage 语义。那部分规范化工作应放到接下来的 J / K canonicalization 一并处理。